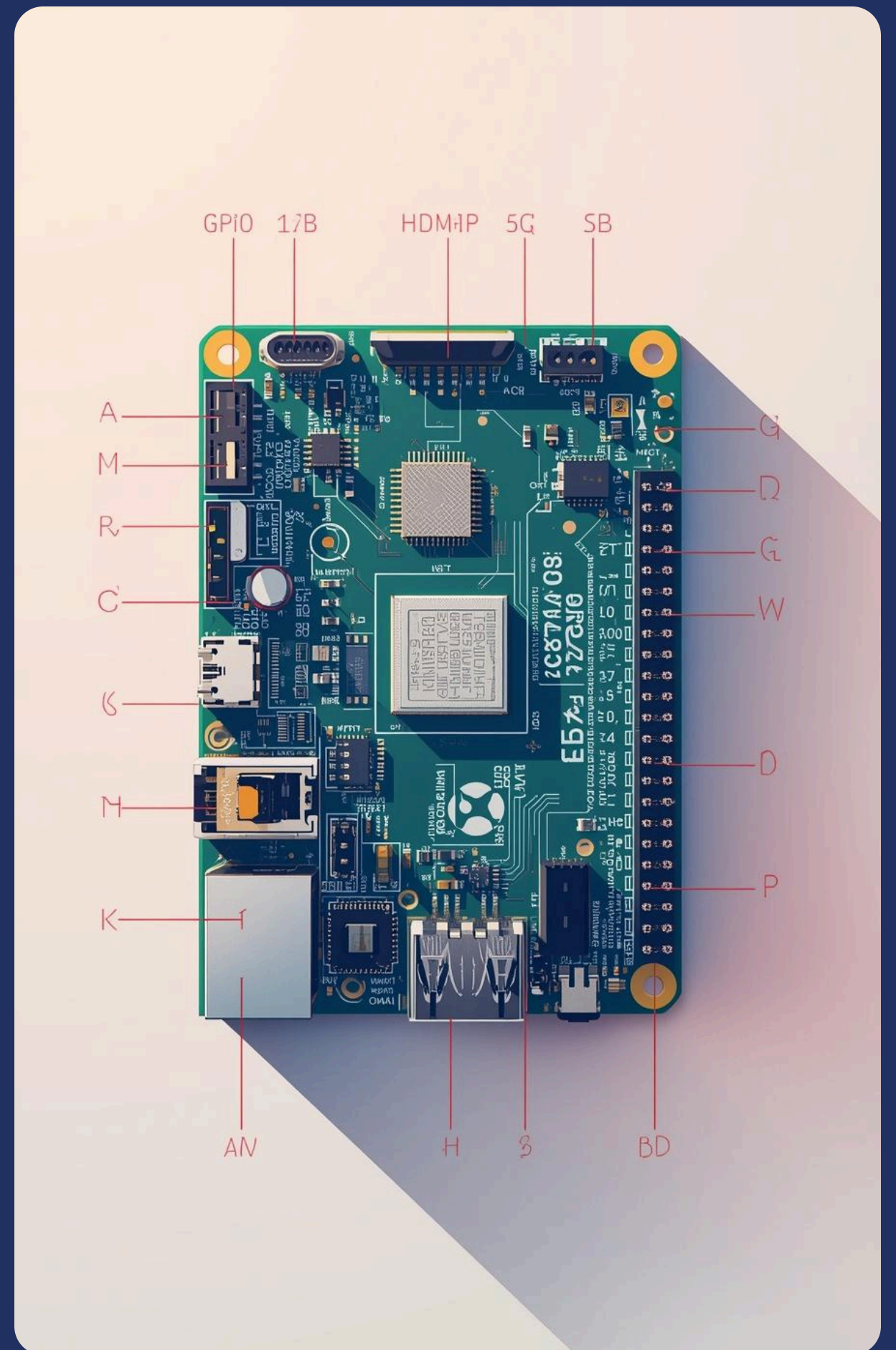
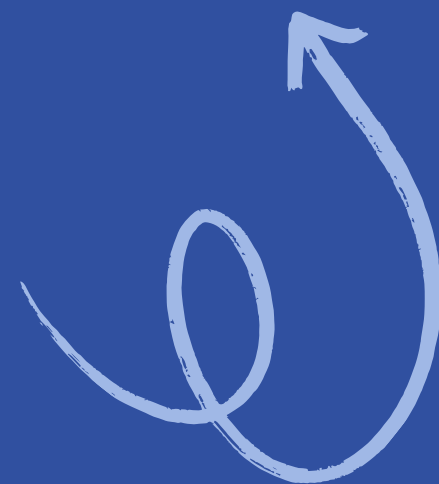


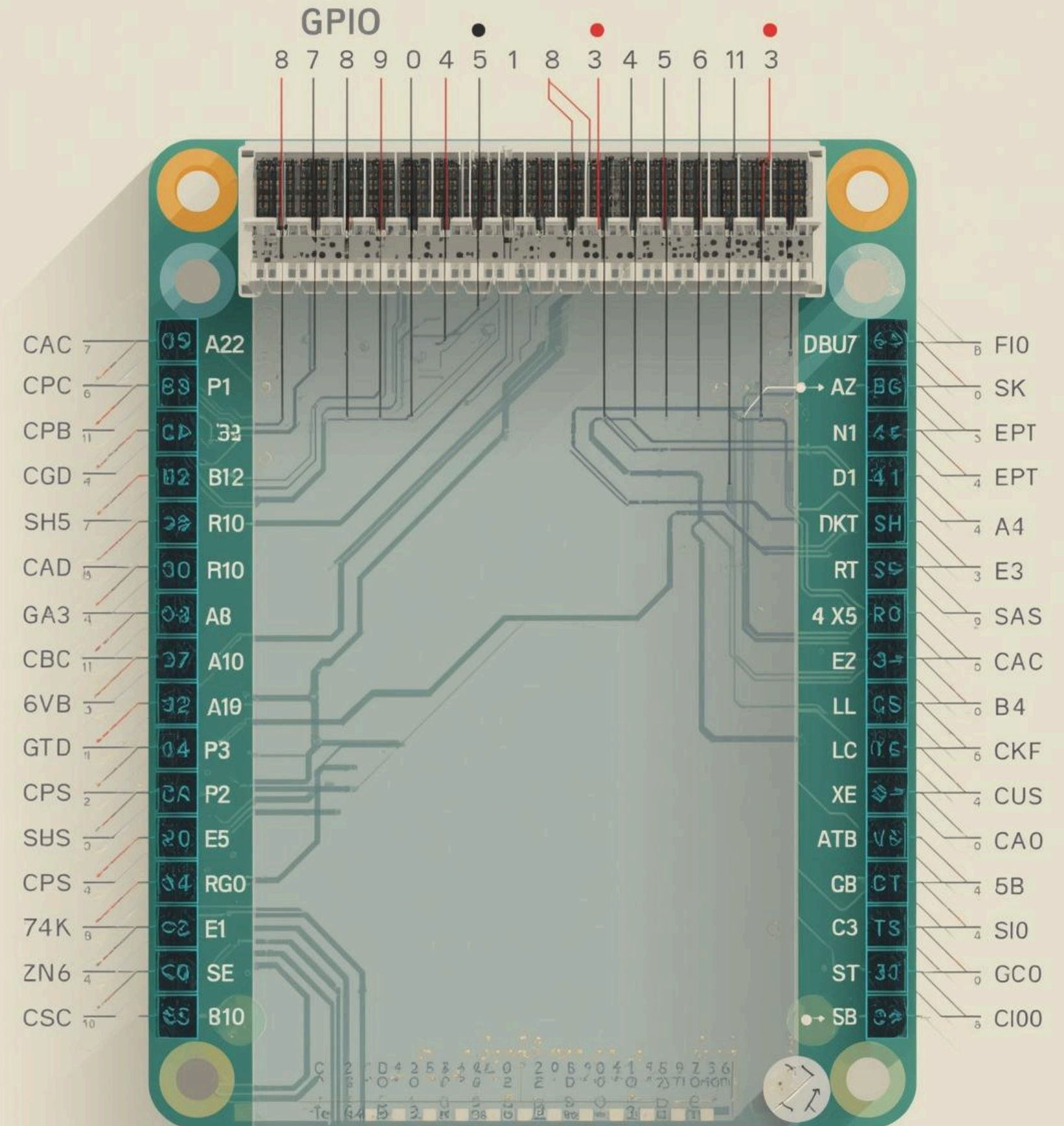
Interfacing Sensors with Raspberry Pi

Presented by Sushant Niraula

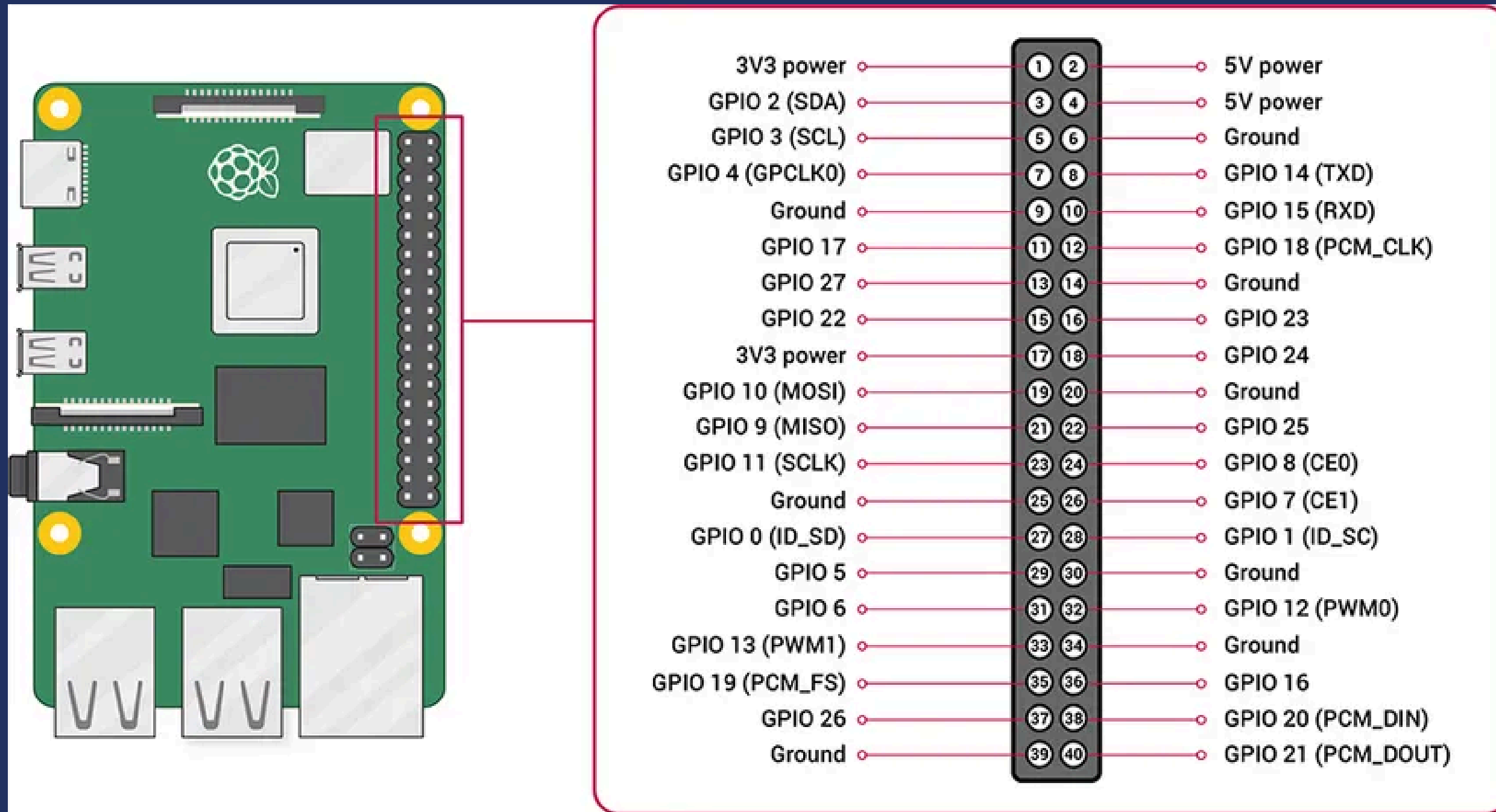


Understanding Raspberry Pi GPIO Architecture

The **GPIO architecture** of Raspberry Pi is essential for interfacing with sensors. It allows for both input and output operations, making it a versatile tool for various electronics projects.



What is GPIO?



General Purpose Input Output.

The Difference:

- Normal Computers: USB, HDMI, Audio (Fixed purpose ports).
- Raspberry Pi: 40 Metal Pins that you decide what they do.

Two Modes:

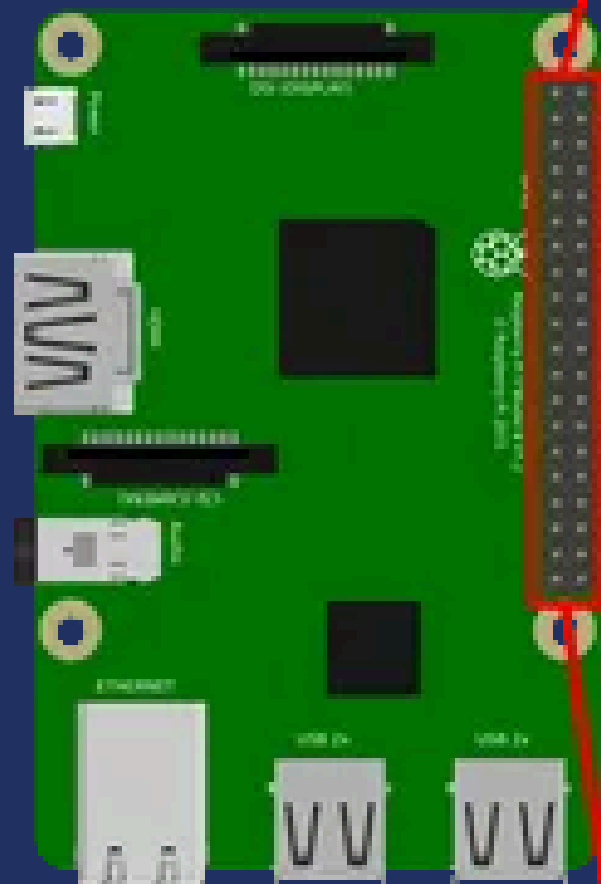
- Input: Reading data (Sensors, Buttons, Switches).
- Output: Sending voltage (LEDs, Motors, Buzzers).

The 40-Pin Header (The Map)

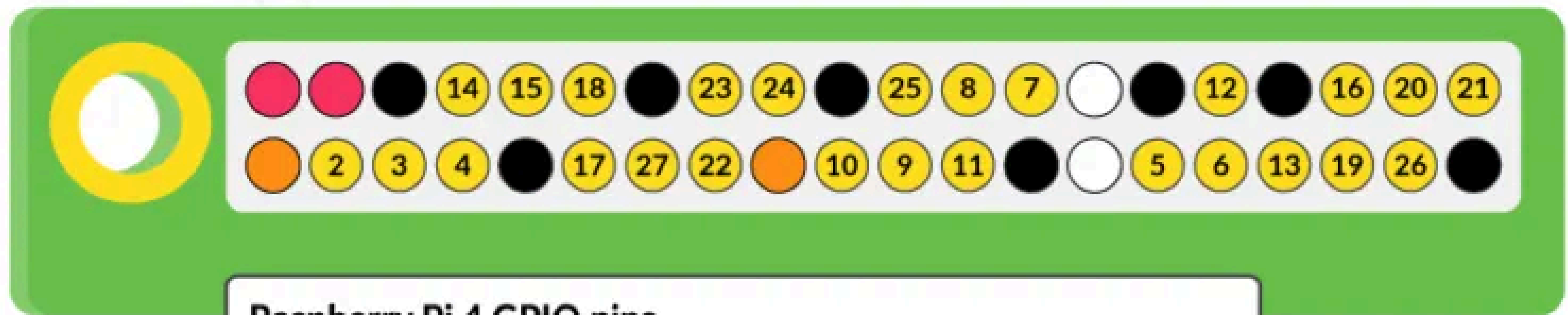
40 pins total (2 rows of 20).

Pin Types:

- Power Pins (Red): 5V and 3.3V (Always ON).
Used to power sensors.
- Ground Pins (Black): GND (0V). The return path for electricity.
- GPIO Pins (Green): The programmable pins (Input/Output).
- Special Pins: ID_SD / ID_SC (Do not touch these; used for HATs).



3.3V PWR	1		2	5V PWR
GPIO2 (SDA1 , I2C)	3		4	5V PWR
GPIO3 (SCL1 , I2C)	5		6	GND
GPIO4 (GPIO_GCLK)	7		8	(UART_TXD0) GPIO14
GND	9		10	(UART_RXD0) GPIO15
GPIO17 (GPIO_GEN0)	11		12	(GPIO_GEN1) GPIO18
GPIO27 (GPIO_GEN2)	13		14	GND
GPIO22 (GPIO_GEN3)	15		16	(GPIO_GEN4) GPIO23
3.3V PWR	17		18	(GPIO_GEN5) GPIO24
GPIO10 (SPI0_MOSI)	19		20	GND
GPIO9 (SPI0_MISO)	21		22	(GPIO_GEN6) GPIO25
GPIO11 (SPI0_CLK)	23		24	(SPI_CE0_N) GPIO8
GND	25		26	(SPI_CE1_N) GPIO7
ID_SD (I2C EEPROM)	27		28	ID_SC (I2C EEPROM)
GPIO5	29		30	GND
GPIO6	31		32	GPIO12
GPIO13	33		34	GND
GPIO19	35		36	GPIO16
GPIO26	37		38	GPIO20
GND	39		40	GPIO21



Raspberry Pi 4 GPIO pins

● GPIO ● Ground ● 3.3v ● 5v ○ EEPROM
Advanced use only!

The Confusion – "Physical" vs "BCM"

Physical Numbers (1-40): Simply counting pins from top-left to bottom-right. Easy to find, but software rarely uses this.

BCM Numbers (GPIO Name): Based on the Broadcom chip inside.

Example: Physical Pin 11 is actually GPIO 17.

The Rule: In Python (gpiozero), we ALWAYS use the BCM (GPIO) Number.

Tip: Keep a "Pinout Card" or image open on your screen at all times.

Safety Rule #1 – The Voltage Trap (3.3V vs 5V)

The Fact: Raspberry Pi GPIO pins work at 3.3 Volts.

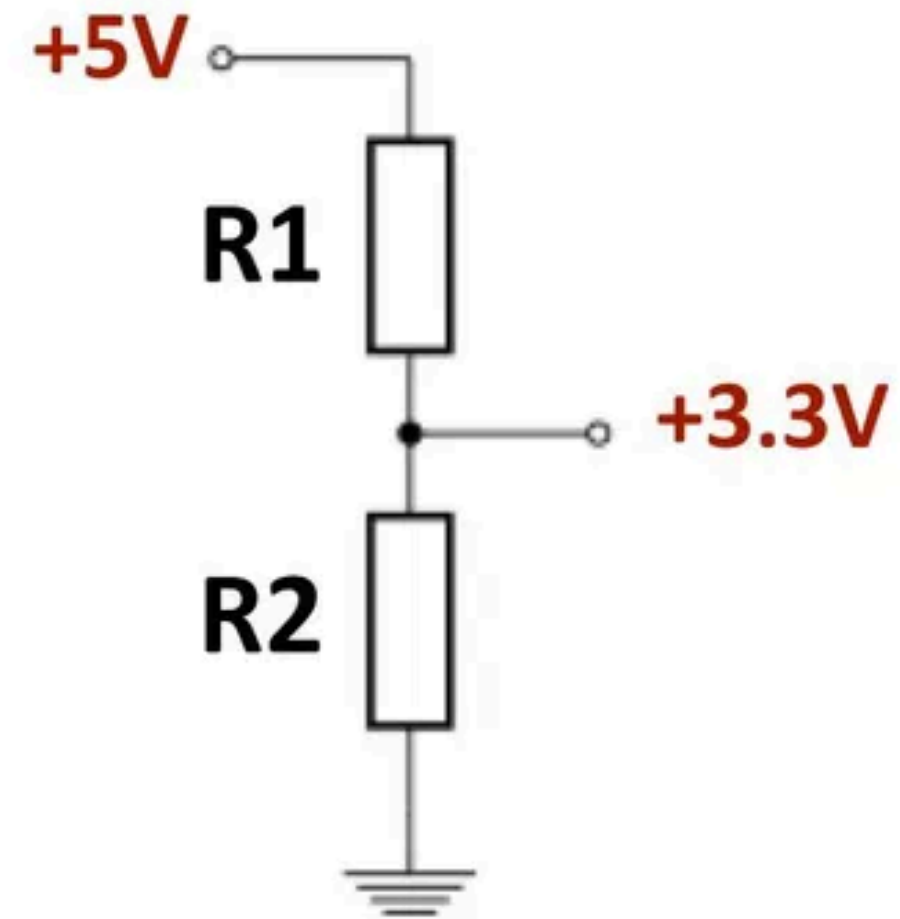
The Danger: Many sensors (and Arduinos) output 5 Volts.

The Scenario:

- Student connects an Ultrasonic Sensor (HC-SR04) Echo pin (5V) directly to Pi GPIO.
- Result: The 5V rushes into the 3.3V chip. The CPU overheats instantly. Game Over.

Use a "Logic Level Converter" or a Voltage Divider (Resistors) to drop 5V down to 3.3V.

5V to 3.3V Voltage Divider



Safety Rule #2 – Power Protocol

Modify ONLY when Dead: NEVER add or remove wires while the Pi is ON.

- Why? If a loose wire accidentally brushes against a 5V pin and a GPIO pin simultaneously, the chip fries.

The "Ground First" Habit:

- Always connect your GND wire first. It establishes a common reference point for the circuit.

Check Twice, Power Once:

Trace your wires with your finger before plugging in the USB-C cable.

Safety Rule #3 – Current Limits

Limit: A single GPIO pin can only provide ~16mA (Milliamps).

What works: LEDs, small buzzers, transistors.

What FAILS: Motors, Pumps, Solenoids, Relay Coils.

Scenario: Connecting a DC motor directly to Pins.

Result: The motor tries to pull 500mA. The Pi pin burns out.

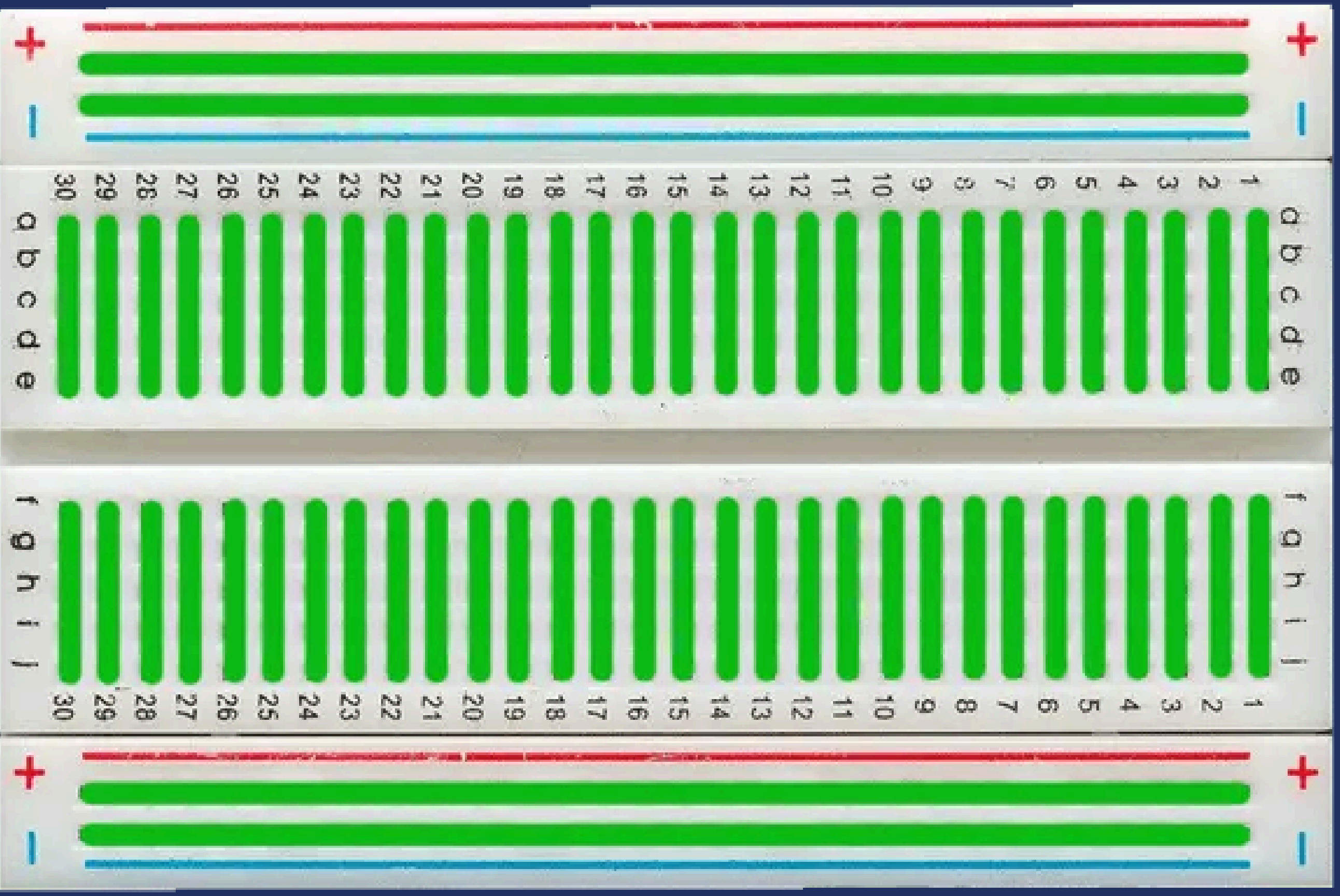
The Fix: Use a Motor Driver (L298N) or a Transistor to switch the heavy load.

Short Circuits (The "Blue Smoke")

Connecting Power (+) directly to Ground (-) with no resistance.

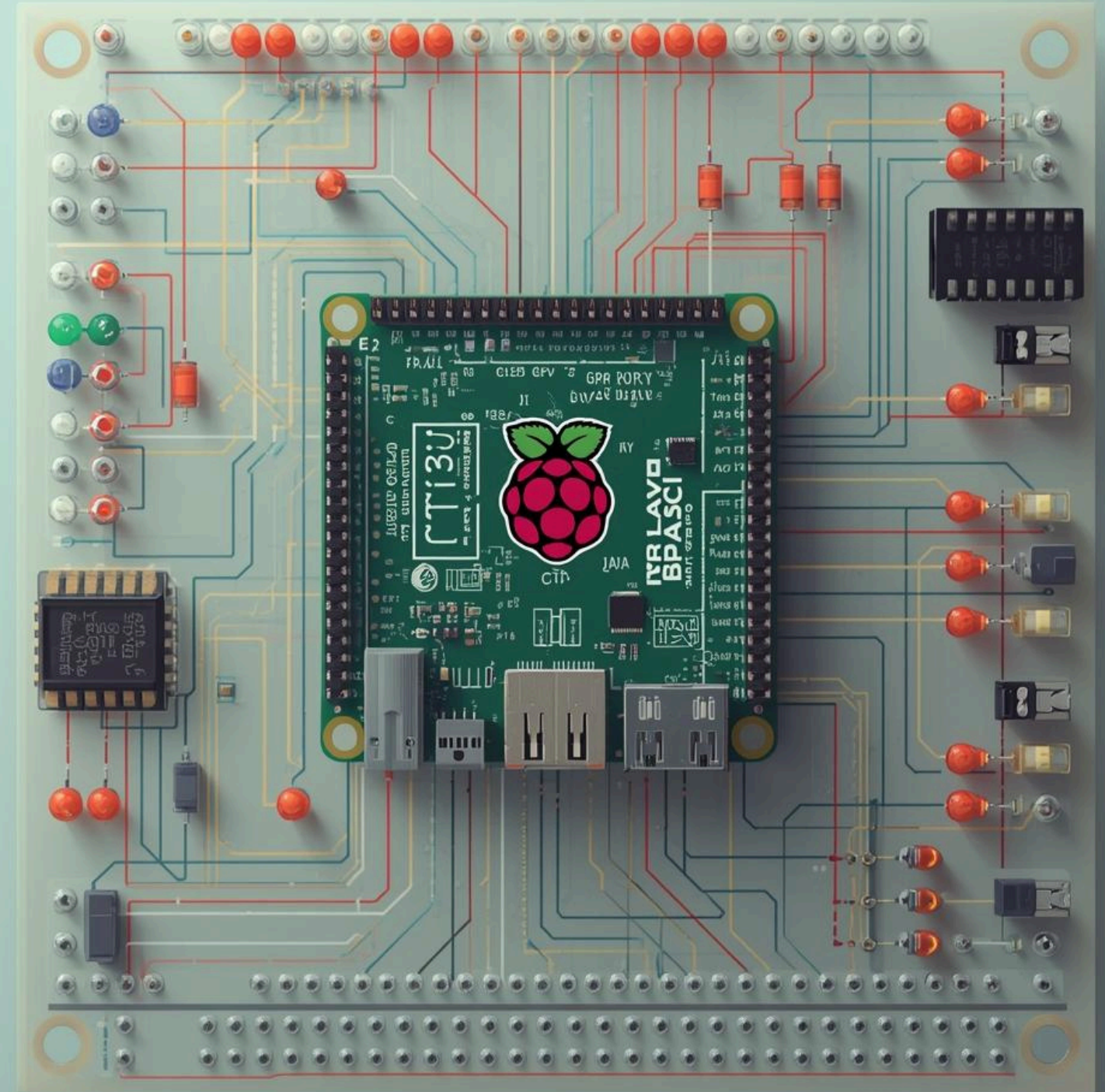
Common Mistakes:

1. Resting the Pi on a metal surface (table leg, coin, laptop chassis).
2. Touching the bottom of the board with a ring or watch.
3. Poking GPIO pins with a screwdriver while running.
4. Protection: Always keep the Pi in a case or on a piece of wood/paper.

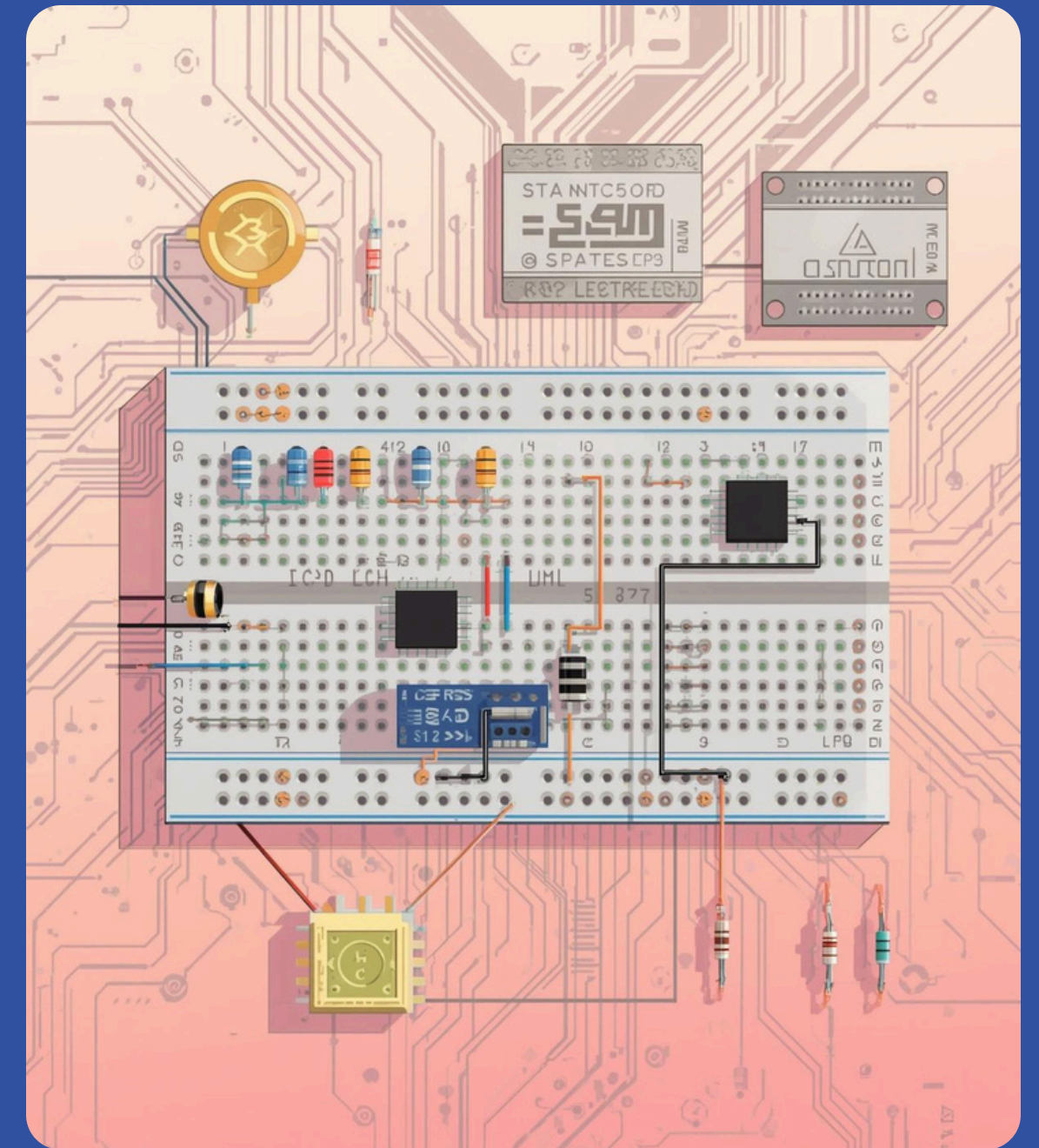
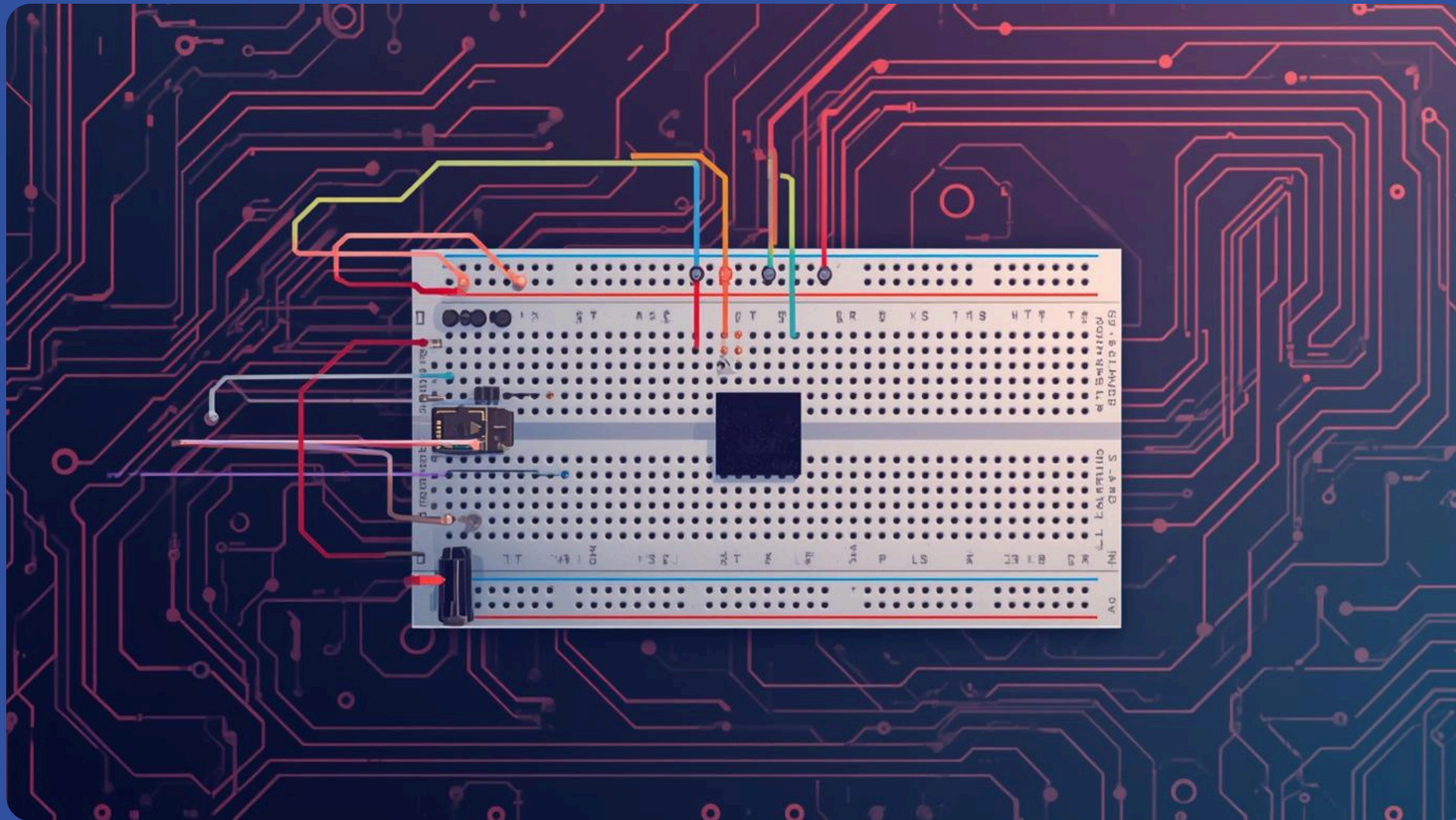


Digital I/O with Raspberry Pi

This section covers how to control **digital signals** using Raspberry Pi's GPIO pins, enabling interaction with various electronic components by sending and receiving digital inputs and outputs through programming.

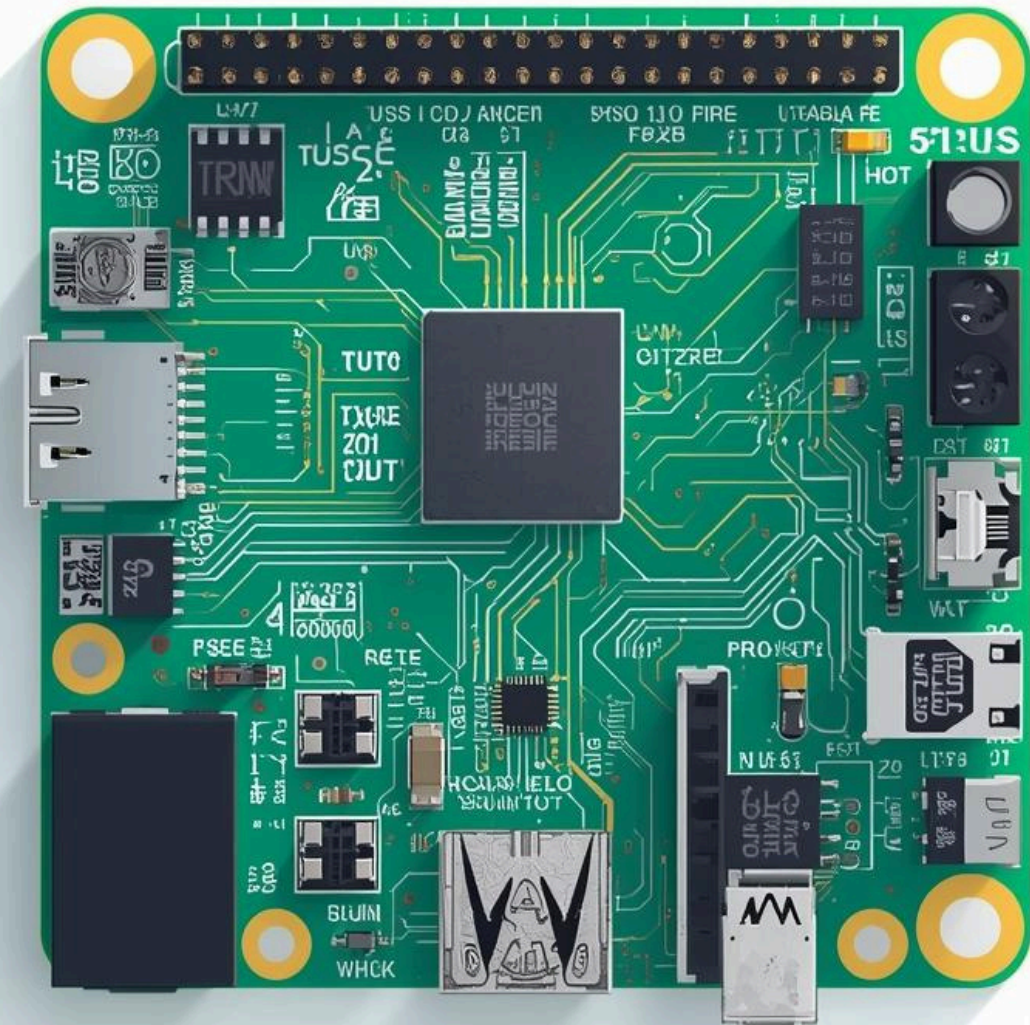


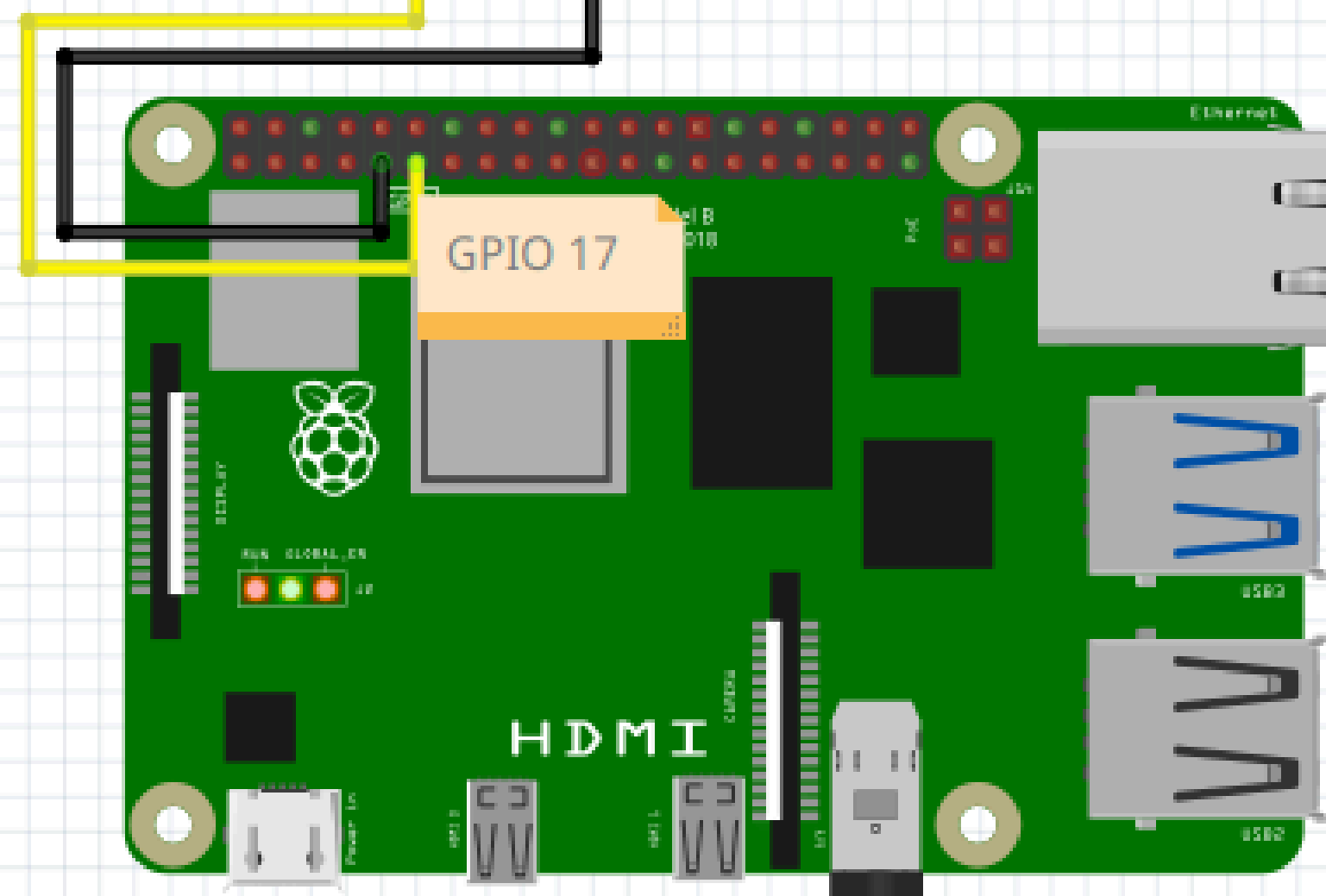
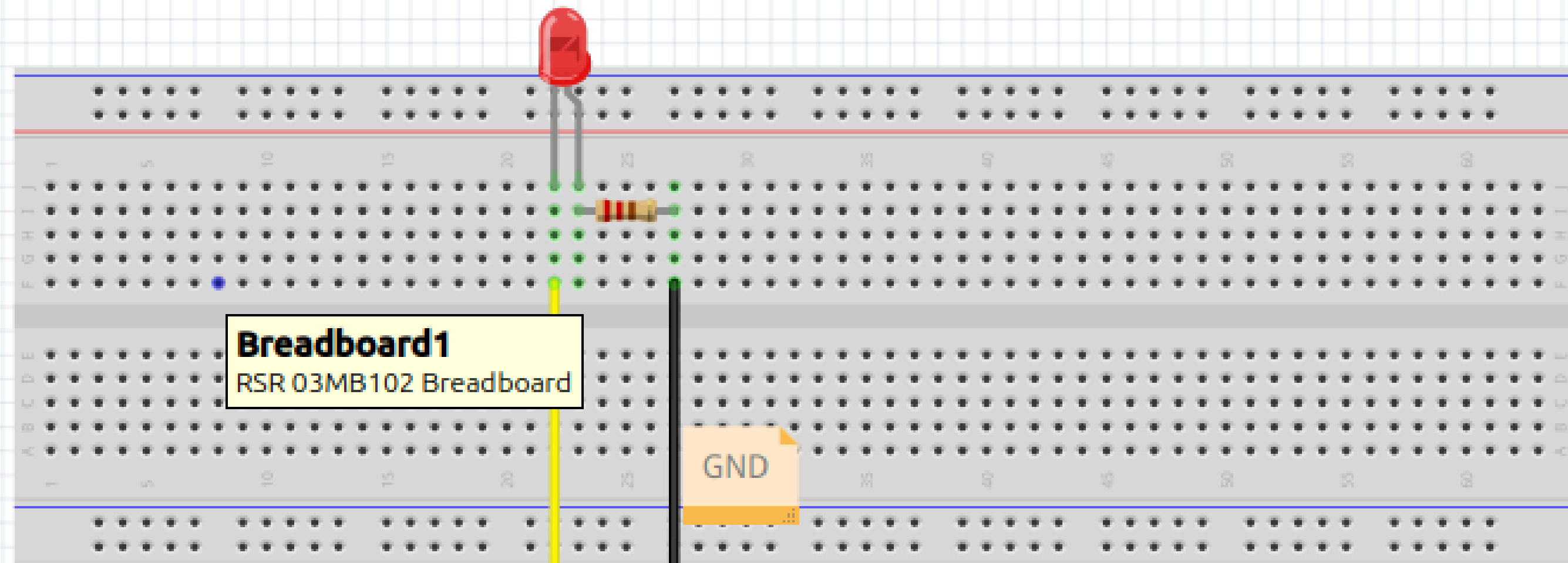
Interfacing Sensors and Components



Python Programming for GPIO Control

Learn how to utilize **Python** to control hardware components on the Raspberry Pi using the RPi.GPIO library. This enables seamless interaction with various sensors and devices through digital signals.





zing

```
from gpiozero import LED
from time import sleep

# Define the object "red" connected to GPIO 17
red = LED(17)

print("Light On!")
red.on()          # Turns the pin to 3.3V
sleep(2)          # Pauses code for 2 seconds (Light stays on)

print("Light Off!")
red.off()         # Turns the pin to 0V
```



```
from gpiozero import LED
from signal import pause # Keeps the program running

red = LED(17)

print("Blinking running in background...")
# Blink: On for 0.5s, Off for 0.5s
red.blink(on_time=0.5, off_time=0.5)

# Essential! Keeps the script alive so the blinking continues
pause()
```

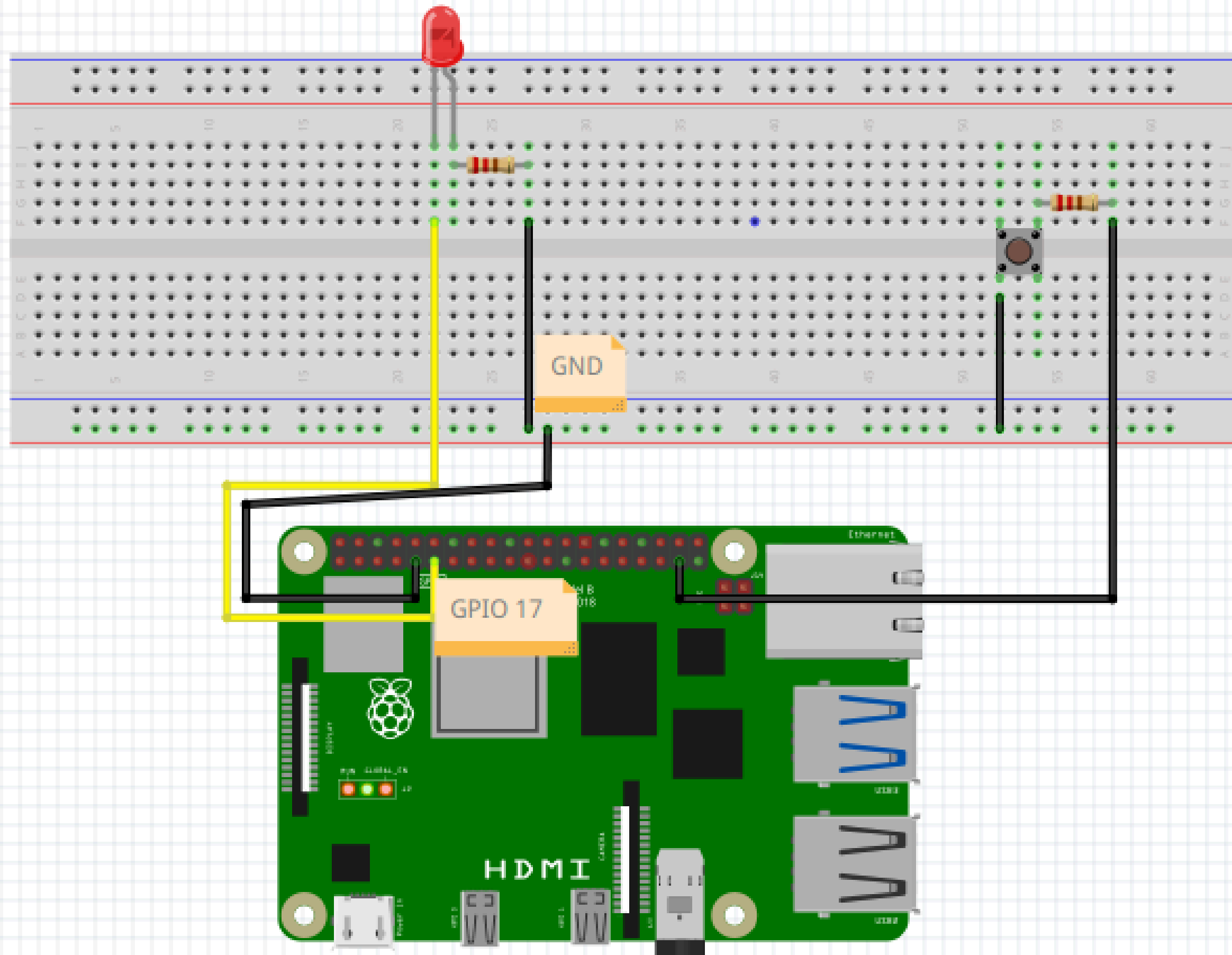
```
from gpiozero import PWMLED
from time import sleep

led = PWMLED(17)

# Set brightness using .value (0.0 to 1.0)
led.value = 0.5 # 50% Brightness
sleep(2)

# Fading Loop
while True:
    for brightness in range(0, 100):
        led.value = brightness / 100.0 # Convert 50 to 0.5
        sleep(0.01)
```

zing



```
from gpiozero import Button
from time import sleep

# Define button on GPIO 2
# gpiozero automatically enables the internal Pull-Up Resistor!
btn = Button(2)

while True:
    if btn.is_pressed:
        print("Button is PRESSED!")
    else:
        print("Button is RELEASED")

    sleep(0.1) # Small delay to stop the CPU from working too hard
```



```
from gpiozero import Button
from signal import pause

def say_hello():
    print("Hello World!")

def say_goodbye():
    print("Goodbye!")

btn = Button(2)

# Assign functions to events (Note: No parentheses '()' on function names!)
btn.when_pressed = say_hello
btn.when_released = say_goodbye

print("Waiting for press...")
pause() # Keep the program running
```

```
from gpiozero import LED, Button
from signal import pause

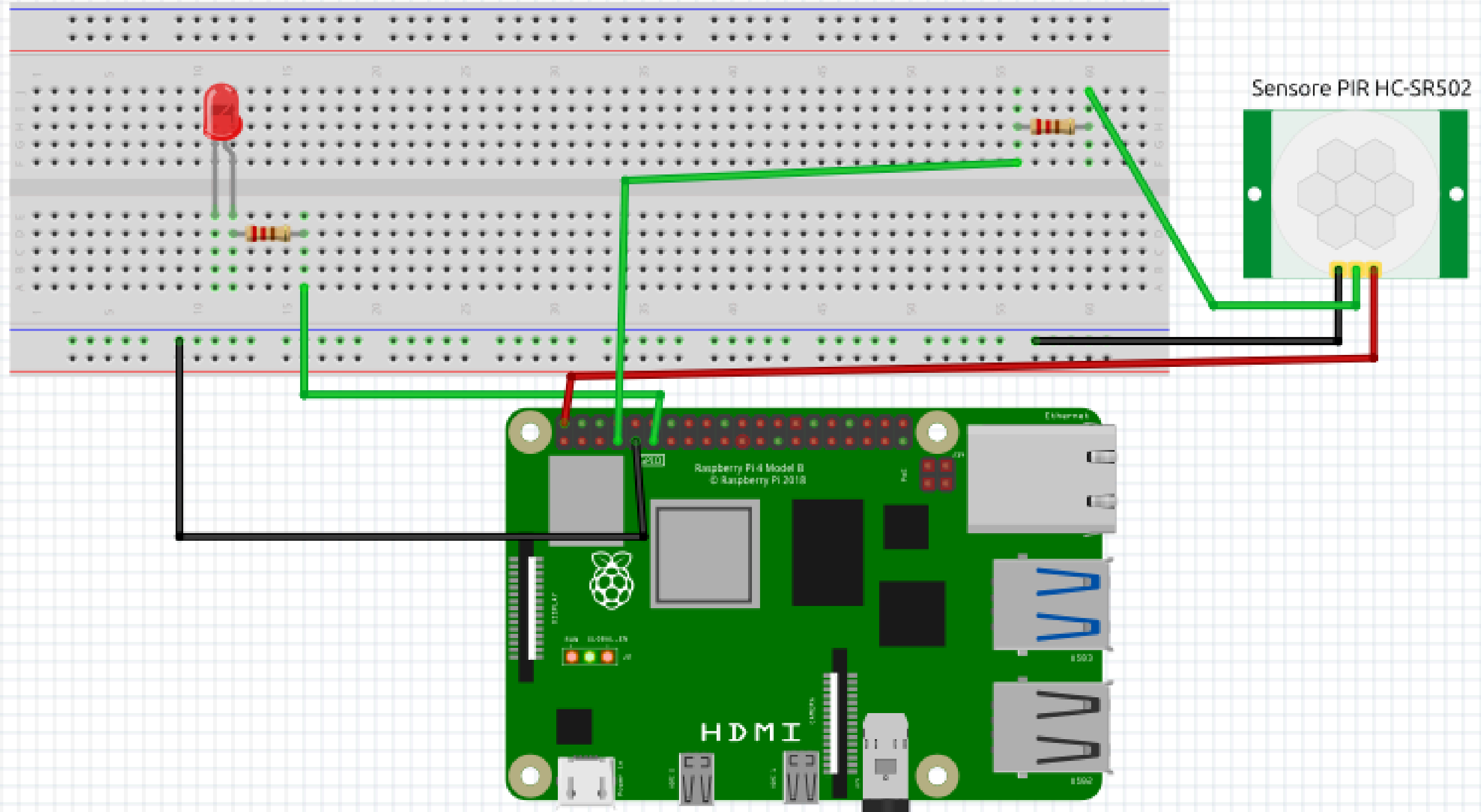
led = LED(17)
btn = Button(2)

# Method 1: Direct Link (Light stays on only while button is held)
# led.source = btn

# Method 2: Toggle (Click to On, Click again to Off)
btn.when_pressed = led.toggle

pause()
```





```
from signal import pause

# Define the sensor on GPIO 4
pir = MotionSensor(4)

def motion_detected():
    print("ALARM! Intruder detected!")

def motion_stopped():
    print("Area is clear.")

# Assign events
print("System Armed...")
pir.when_motion = motion_detected
pir.when_no_motion = motion_stopped

pause()
```



```
from gpiozero import MotionSensor, LED
from signal import pause

pir = MotionSensor(4)
led = LED(17)

# One line of magic code!
# This ties the LED state directly to the Sensor state
led.source = pir

pause()
```

1. Installing GPIO Zero
2. Basic Recipes
3. Advanced Recipes
4. Configuring Remote GPIO
5. Remote GPIO Recipes
6. Pi Zero USB OTG
7. Source/Values
8. Command-line Tools
9. Frequently Asked Questions
10. Backwards Compatibility
11. Migrating from RPi.GPIO
12. Contributing
13. Development
14. API - Input Devices
15. API - Output Devices
16. API - SPI Devices
17. API - Boards and Accessories

gpiozero

A simple interface to GPIO devices with [Raspberry Pi](#), developed and maintained by [Ben Nuttall](#) and [Dave Jones](#).

About

Component interfaces are provided to allow a frictionless way to get started with physical computing:

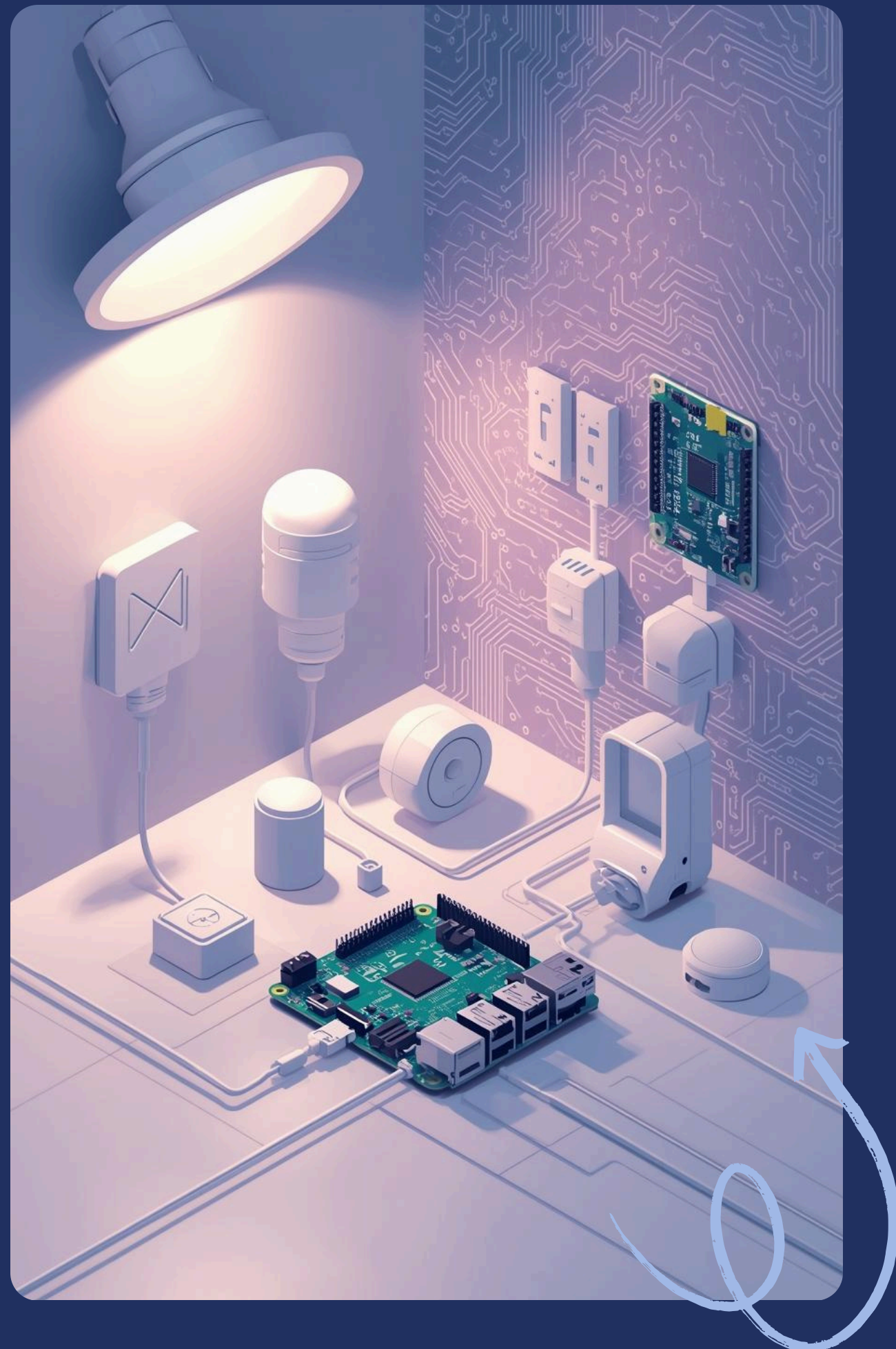
```
from gpiozero import LED
from time import sleep

led = LED(17)

while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

With very little code, you can quickly get going connecting your components together:

<https://gpiozero.readthedocs.io/en/stable/>



Continuing the Journey

Explore more Raspberry Pi projects

Serial Communication with Arduino To use GPIO pins in arduino and control it using PI. We will learn tomorrow.